

Abstract

Accessing legal and administrative regulations within academic institutions is often difficult for students and researchers. Legal texts are typically written in complex language and distributed across multiple documents. This work presents the design of a **multilingual legal advisory chatbot and NLP assistant** that assists users in navigating legal procedures and institutional regulations

example:

A PhD student whose supervisor dies during the thesis must follow specific administrative procedures defined by university regulations.

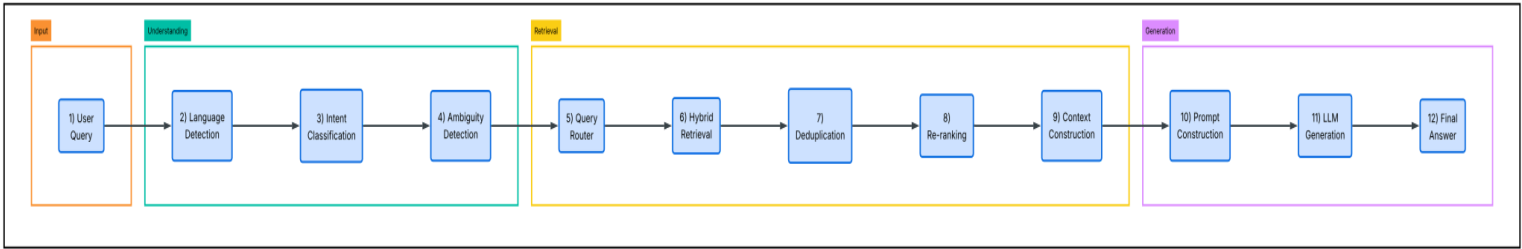
Traditional search methods require manually reading many documents. Conversational AI systems provide a more accessible solution by allowing users to ask questions in natural language.

However, purely generative models may produce **hallucinated legal information**. Modern systems address this issue using **Retrieval-Augmented Generation (RAG)**, where responses are grounded in external documents.

1.Objective

- implement **hybrid re-ranker retrieval over legal documents and nlp_knowledge**
- provide **grounded responses using RAG**
- evaluate system performance using **retrieval and generation metrics**

1.END to END pipeline:



2.Language Detection:

The system supports **Arabic, French, and English** using a hybrid detection strategy, uses two techniques combined.

1/Unicode script analysis:

Arabic characters are detected using a regular expression a specialized sequence of characters defining pattern:

```
[ \u0600-\u06FF\u0750-\u077F\u08A0-\u08FF ]
```

The Arabic ratio is computed as:

$$R_{arabic} = \frac{\text{number of Arabic characters}}{\text{total alphabetic characters}}$$

Decision rule:

$$R_{arabic} \geq 0.30 \Rightarrow Arabic$$

2/Statistical language detection:

If the text is not Arabic, the system uses the library: **langdetect**

This library implements character n-gram(a sequence of **n characters**.) language models.

The predicted language corresponds to the maximum likelihood estimate.

$$L^* = \underset{L}{\operatorname{argmax}} P(L|T)$$

The best answer (L^*) is the one with the highest probability of being correct, given the provided text (T).

3.Intent Classification:

The system defines multiple query intents:

Intent	Purpose
legal_query	legal regulations
platform_query	platform usage
document_query	uploaded documents
conceptual_question	NLP / AI concepts
metadata_query	User information
general_knowledge	LLM knowledge

1.Pattern-Based Classification:

The classifier uses regex pattern banks.

Example:

Legal query patterns:Example match: Is this legal?

```
LEGAL_PATTERNS = [  
    r"\blegal\b",  
    r"\blaw\b",  
    r"\bregulation\b", ]
```

1.2. Confidence-Based Scoring:

multi-intent scoring.

Each intent receives a score. then The system selects the **highest score**.

Scoring Formula

From the code:

```
count = sum(1 for p in patterns if p.search(text))
```

If at least one pattern matches:

$$Score = Base + BonusScore$$

Where:

Base score depends on intent

Bonus:

$$Bonus = (matches - 1) \times 0.03$$

1.3 Ambiguity Detection:

system detects ambiguous classifications.

$$\Delta = Score_{top} - Score_{second}$$

If $\Delta < 0.15$, the classifier considers the query **ambiguous**. Confidence is reduced:

confidence = min(top_score, 0.60).

1.4 LLM Disambiguation:

When the classifier is uncertain, your system uses an **LLM fallback**. The LLM is given a user query along with a predefined set of candidate intents (e.g., *user_query*, *legal_query*, *platform_query*). It then analyzes the query and selects the intent that best matches its meaning.

For example, given the question “*What are the legal requirements for data privacy?*”, the model identifies *legal_query* as the most relevant intent.

This approach combines a **symbolic component** (the fixed list of possible intents) with the **reasoning capabilities of the LLM**, resulting in a hybrid classification method.

4.Query Routing:

After identifying the intent, The routing mechanism directs queries to different databases.

Intent	Data Source
conceptual_question	Qdrant
platform_query	PostgreSQL / Elasticsearch
legal_query	Qdrant legal documents
document_query	Qdrant user documents
bug_query	Qdrant knowledge
metadata_query	PostgreSQL
general_knowledge	LLM directly

the query must be routed to the appropriate knowledge source.

5.Query Representation:

Each user query is converted into a vector representation. both queries and documents are converted into vector representations using an embedding model.

Let; $q=(q_1,q_2,...,q_n)$ be the query vector, and $d=(d_1,d_2,...,d_n)$ be a document vector.

These vectors are generated using a multilingual transformer model.

6.Hybrid re ranking Retrieval:

6.1.Semantic Similarity

To measure the similarity between a query and a document, the system uses **cosine similarity**.

The cosine similarity between vectors q and d is defined as:

$$similarity(q, d) = \frac{q \cdot d}{\|q\| \cdot \|d\|}$$

where

$$q \cdot d = \sum_{i=1}^n q_i d_i$$

Values closer to **1** indicate higher semantic similarity.

The final score of a document is calculated using weighted scoring:

$$score_{weighted} = similarity(q, d) \times w_{source}$$

Where: This prioritizes important sources such as official legal texts.

Source	Weight
Platform docs	1.10
Legal documents	1.05
Knowledge base	1.00

6.2 Deduplication:

To avoid redundant information, the system removes near-duplicate document chunks.

This is performed using **Jaccard similarity**.

For two document sets A and B:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

If

$$J(A, B) \geq 0.85$$

the documents are considered duplicates.

6.3 Reranking:

After retrieval and deduplication, candidate documents are reranked using cosine similarity.

The reranking score is defined as:

$$score_{rerank} = \frac{q \cdot d}{\|q\| \|d\|}$$

This step improves the final ranking quality then return only top 5 useful chunks

7.Context Construction:

This module prepares the prompt sent to the language model.

The context includes:

- retrieved legal documents
- relevant platform knowledge
- conversation history

8. Response Generation Module

The final response is generated by a **large language model**.

Prompt->LLM Processing->Token Generation->Final Answer

9. Evaluation Methodology:

Evaluation is performed at two levels:

1. Retrieval performance
2. Response quality

Retrieval Evaluation

This evaluates **how well the system retrieves legal documents**.

Metrics:

Precision@k

$$Precision@k = \frac{\text{relevant documents retrieved}}{k}$$

Example:

If 4 relevant documents appear in top 5:

$$Precision@5 = 4/5 = 0.8$$

Recall@k

$$Recall@k = \frac{\text{relevant documents retrieved}}{\text{total relevant documents}}$$

This measures **coverage of relevant legal information**.

Mean Reciprocal Rank (MRR)

MRR measures how quickly the system finds the correct document.

$$MRR = \frac{1}{N} \sum_{i=1}^N \frac{1}{rank_i}$$

Where:

- rank_i is the position of the first correct answer.

Higher MRR = better system.

5.2 Response Quality Evaluation

These metrics evaluate **the generated answers**;

BERTScore (Best for your case)

A better metric for LLM evaluation.

Instead of word overlap, it measures **semantic similarity** using embeddings.

$$BERTScore = \frac{\sum similarity(token_i, token_j)}{N}$$

This is **better for multilingual systems**.

metrics for chatbot

Use **three metrics together**:

Metric	Purpose
Precision@k	retrieval quality
MRR	ranking quality
BERTScore	answer semantic accuracy

5.3 Human Evaluation

Automatic metrics are not enough for legal chatbots.

Add **human expert evaluation**.

Evaluation criteria:

Criterion	Description
Accuracy	correct legal information
Completeness	full explanation
Clarity	understandable answer
Source citation	references provided

